

DriveShare: Project Report

This report includes:

URLS, UML Class Diagrams, Database Schema, and User Interface Screenshots.



INTRODUCTION

DriveShare is a standalone platform designed to connect car owners with individuals seeking short-term car rentals. The platform aims to provide a convenient and efficient solution for users to rent personal vehicles directly from their owners.

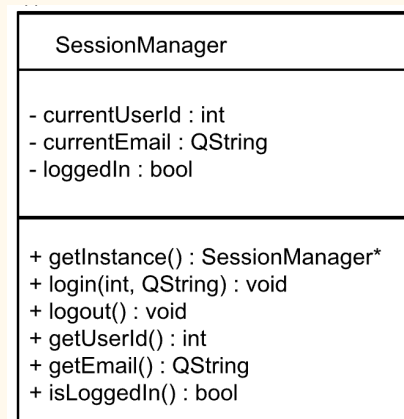
URLS:

- Github Repository URL: <https://github.com/mayanour4/DriveShare.git>
- Video Presentation URL: <https://youtu.be/5sd8b1sIZoo>

UML Class Diagrams:

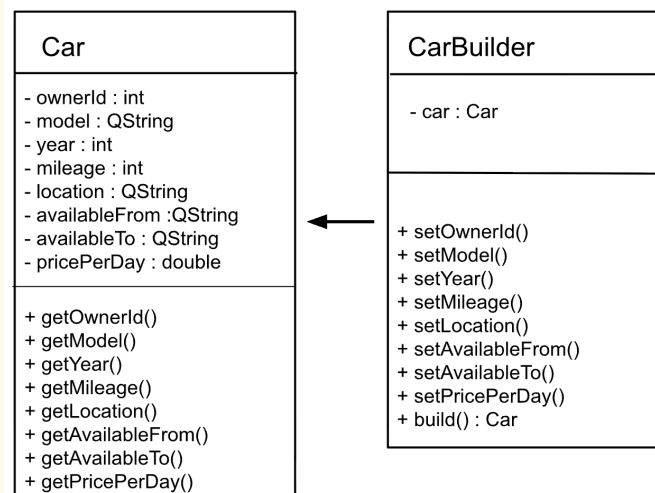
1. Singleton Pattern:

The SessionManager class ensures that only one user session exists throughout the application. It stores the currently logged-in user's information and provides a centralized way to manage authentication state.



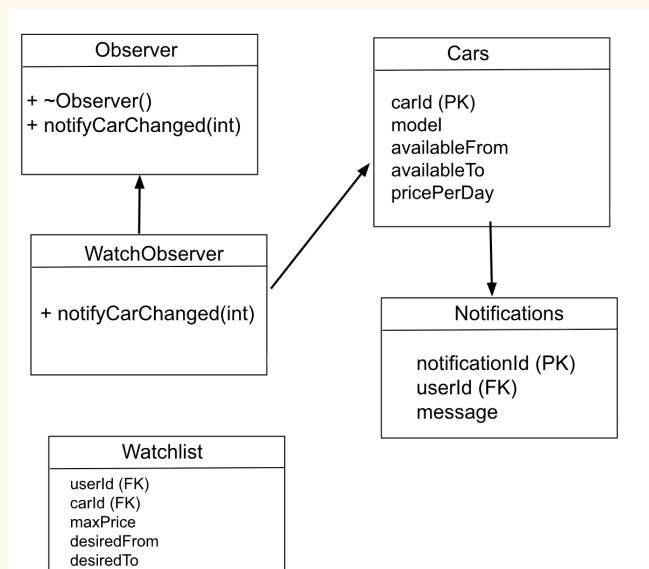
2. Builder Pattern:

The Builder pattern is used to construct car objects step-by-step instead of using a large constructor with many parameters. This makes car listing creation more flexible, readable, and easier to maintain.



3. Observer Pattern:

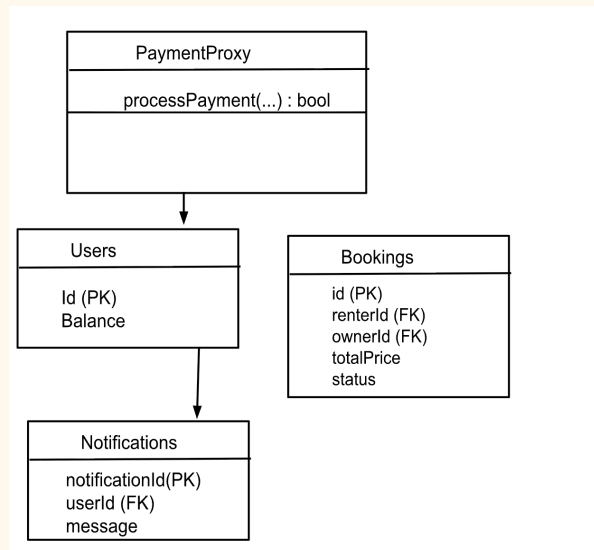
The Observer pattern allows renters to watch cars and receive notifications when their preferences are met. WatchObserver monitors car updates and automatically creates notifications when matching cars become available or meet price requirements.



4. Proxy Pattern:

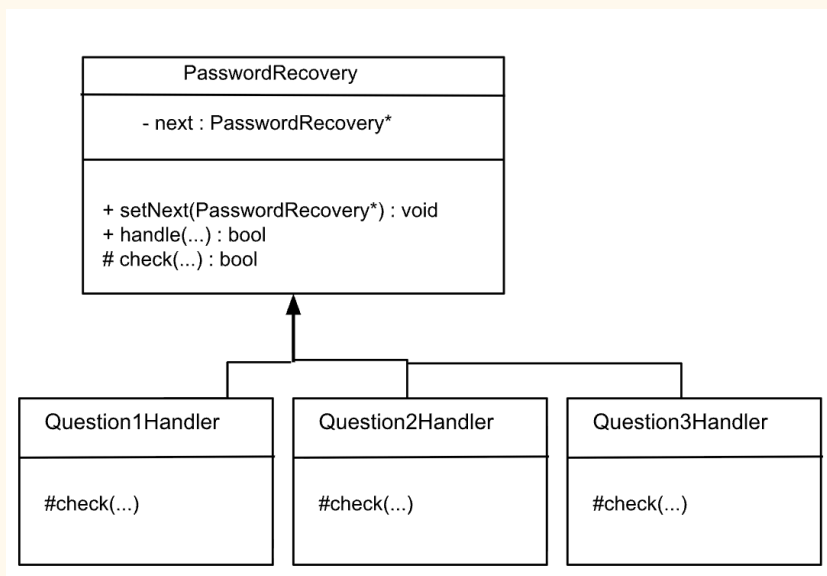
The Proxy pattern controls access to the payment process by validating bookings and balances before completing transactions. PaymentProxy acts as an

intermediary between users and the payment system, ensuring secure and consistent payment handling.



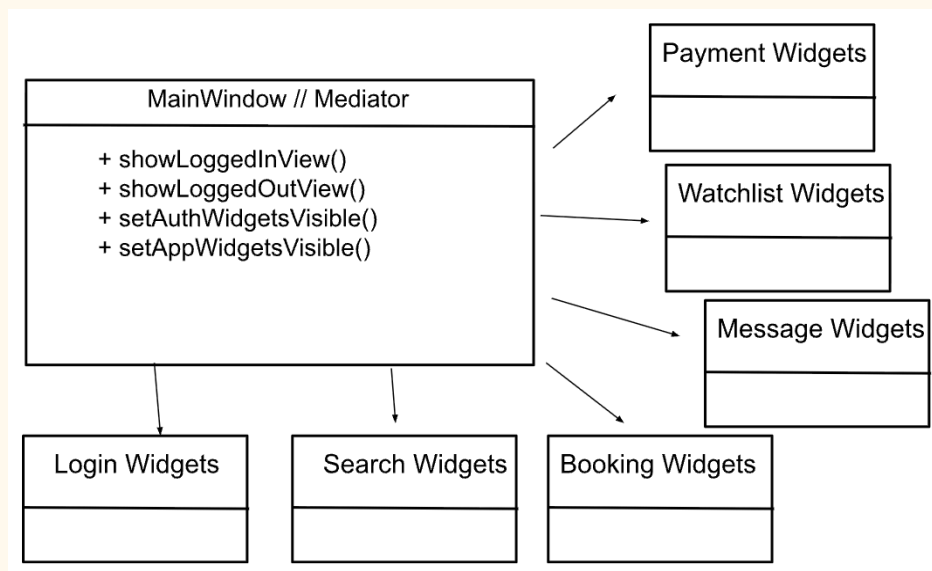
5. Chain of Responsibility Pattern:

The Chain of Responsibility pattern processes password recovery by validating security questions one at a time. Each handler checks a specific answer and passes the request to the next handler only if the current validation succeeds.



6. Mediator Pattern:

The Mediator pattern is implemented through **MainWindow**, which coordinates communication between UI components. Instead of widgets interacting directly with one another, **MainWindow** manages visibility and navigation throughout the application.



Database Schema:

1. Users Table: stores registered users.

Id	unique user ID
email	user's login email
password	user's password
securityQuestion1/2/3	password recovery questions
securityAnswer1/2/3	answers for password recovery
balance	fake money balance for payment

2. Cars Table: stores cars listed by owners.

Id	unique car ID
ownerId	User ID of the owner
model	car model
year	car year
mileage	car mileage
location	Pick up location
availableFrom	start availability date
availableTo	end availability date
pricePerDay	Rental price per day

3. Bookings Table: stores rental reservations.

Id	unique booking ID
carId	Id of the car being rented
renterID	User who rents the car
ownerID	User who owns the car
startDate	Rental start date

endDate	Rental end date
status	Booking status like unpaid or paid
totalPrice	Total rental cost

4. Notifications Table: Stores messages alerts shown to users.

Id	unique notification ID
userId	User receiving the notification
message	Notification text

5. Watchlist Table: Stores the cars that renters are watching.

Id	unique watchlist entry id
userID	Renter watching the car
carID	Watched car
maxPrice	Renter's desired max price
desiredFrom	Desired start date
desiredTo	Desired end date

6. Messages Table: Stores messages between users

Id	unique message id
SenderId	User who sent the message
ReceiverId	User receiving the message
Message	Message text

Functions/Features:

1. User Registration and Authentication:

a. Registration: Success

The diagram illustrates the user registration process. It shows two states of the 'DriveShare Login' form. In the initial state (left), the 'Email' field contains 'jmail@example.com'. In the final state (right), the 'Email' field contains 'mayanour@test.com', and a 'Registration successful!' modal is displayed. The 'Security Questions and Answers' section is also shown, with three questions and their corresponding answers.

DriveShare Login 🚗

Email:

Password:

[Forgot Password](#)

[Login](#) [Register](#)

Security Questions and Answers 🔒

Security question 1:

Answer 1:

Security question 2:

Answer 2:

Security question 3:

Answer 3:

DriveShare Login 🚗

Email:

Password:

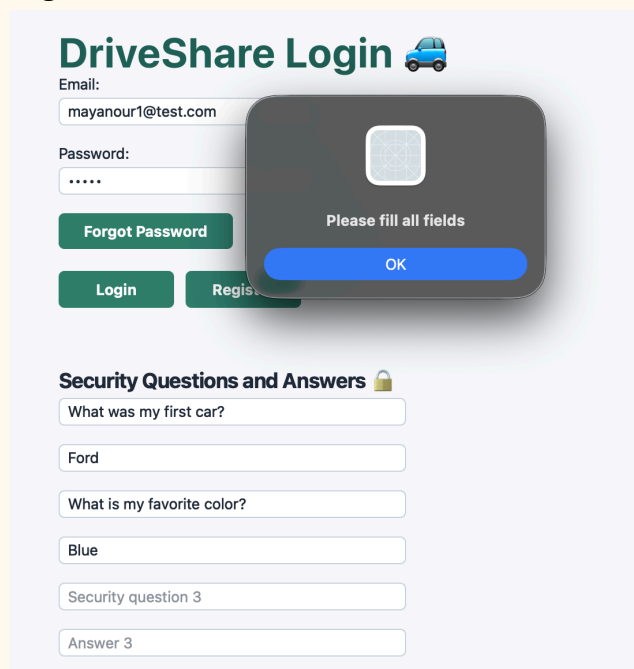
[Forgot Password](#)

[Login](#) [Register](#)

Registration successful!

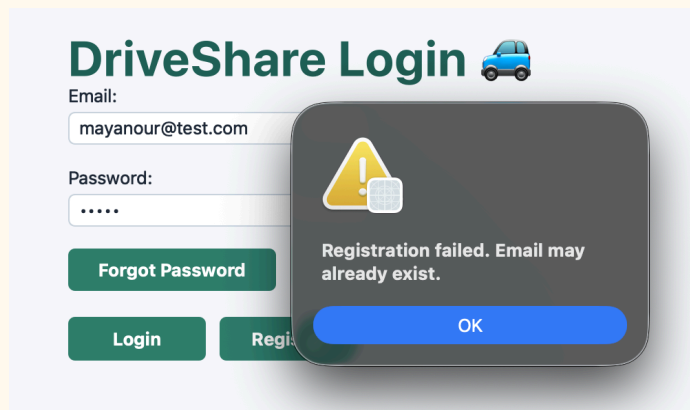
[OK](#)

b. Registration: Set all Fields



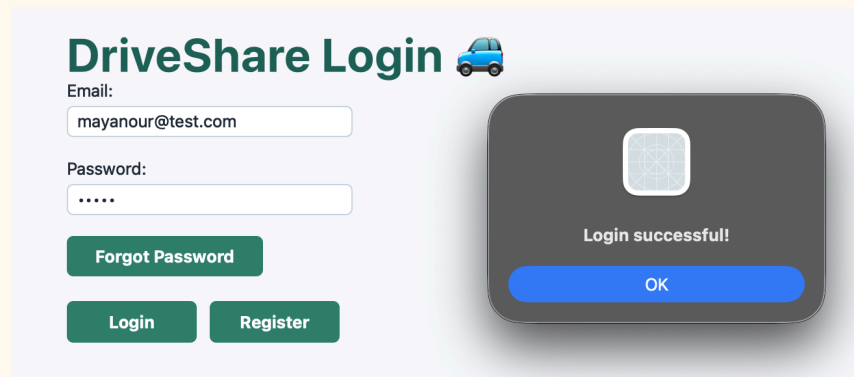
The image shows the 'DriveShare Login' form. The title 'DriveShare Login' is in green with a car icon. Below it are fields for 'Email:' (containing 'mayanour1@test.com') and 'Password:' (with masked dots). There are buttons for 'Forgot Password', 'Login', and 'Register'. A modal dialog box is open in the center, displaying a grid icon and the text 'Please fill all fields' with an 'OK' button. Below the login section is a 'Security Questions and Answers' section with a lock icon, containing three questions and their corresponding answers: 'What was my first car?' (Ford), 'What is my favorite color?' (Blue), and 'Security question 3' (Answer 3).

c. Registration: Account already exists:

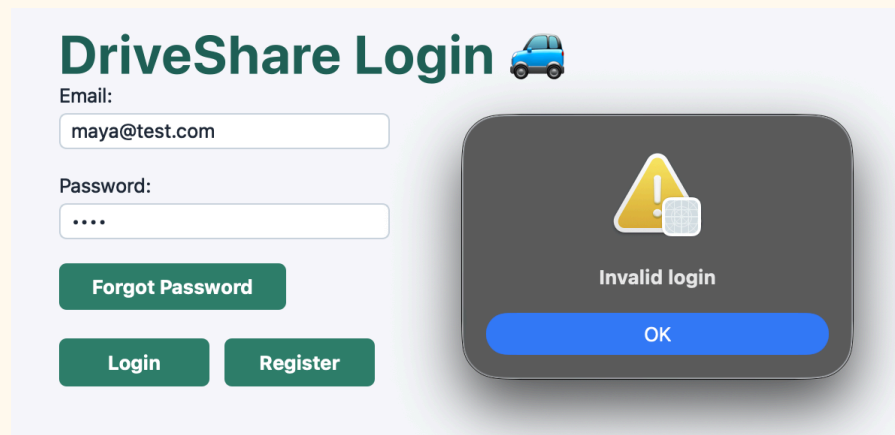


The image shows the 'DriveShare Login' form with the same fields and buttons as in the previous screenshot. A modal dialog box is open, displaying a yellow warning triangle icon and the text 'Registration failed. Email may already exist.' with an 'OK' button.

d. Log in: Success



e. Log in: Fail (Invalid credentials)



2. Car Listing and Management:

a. Listing:

List a Car 🚗

Model:

Year:

Mileage:

Location:

From:

To:

Price/day:

Add Car



Car listed successfully

OK

b. Managing and Updating Listing:

Update Listing 🔄

Car ID:

From:

To:

Price/day:

Update Car



Car listing updated successfully

OK

c. Prevent overlapping rentals:



This car is already booked for those dates

OK

3. Searching and Booking:

a. Search Available Cars:

Search Cars

Location:

From:

To:



Car ID: 2
Model: Ford
Year: 2013
Mileage: 100000
Location: Dearborn
Available: 2026-06-20 to 2026-06-25
Price/day: \$50

b. Watch Cars and Receive Notification:

Watch a Car

Car ID:

Max Price:

From:

To:



Car added to watchlist!



- Watched car is available: Ford
(Car ID: 2)

c. Booking Success:

The screenshot shows a 'Book a Car' form with a green checkmark icon. The form fields are: Car ID: 2, From: 2026-06-20, To: 2026-06-25. A green 'Book' button is at the bottom. To the right, a dark grey modal box displays a car icon, the text 'Booking successful! Booking ID: 1' and 'Total price: \$120', and a blue 'OK' button.

d. Booking Invalid: Own Car

The screenshot shows the same 'Book a Car' form as in (c). However, the modal box on the right displays a car icon and the text 'You cannot book your own car', with a blue 'OK' button.

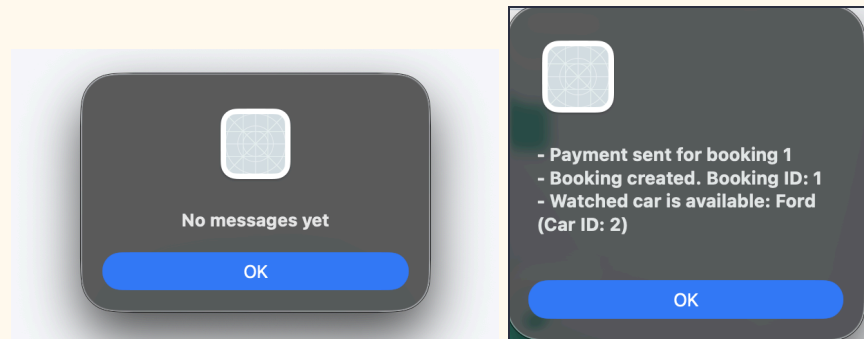
4. Messaging and Communication:

a. Message System:

The screenshot shows a 'Messages' form with a red envelope icon. The form fields are: To Email: mayanour@test.com, Message: hello. There are green 'Send' and 'View Messages' buttons. To the right, a dark grey modal box displays a car icon and the text 'Message sent!', with a blue 'OK' button.

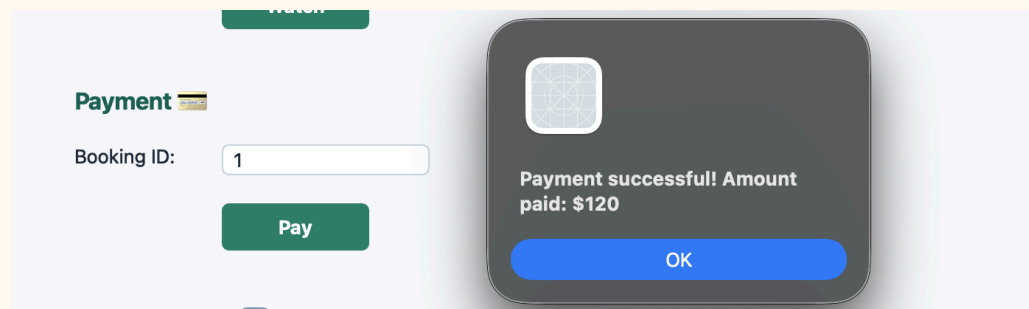
Below this, the same 'Messages' form is shown, but the 'View Messages' button is highlighted with a red box. To the right, a dark grey modal box displays a car icon and the text 'From: maya2@test.com' and 'hello', with a blue 'OK' button. A red arrow points from the 'View Messages' button to the modal box.

b. Other Notifications:



5. Payment:

a. Payment Successful:



b. Invalid Payment: (Already paid)

